

CSE 8803 HW1: Building a Decoder-Only Transformer from Scratch

GT Username: yzhu751

Georgia Institute of Technology – Fall 2026

Abstract

For this project, I built a byte-level BPE tokenizer and a decoder-only language model without using ready-made tokenizer, attention, or Transformer modules. The model includes RMSNorm, rotary position embeddings (RoPE), grouped-query attention (GQA), SwiGLU, tied embeddings, my own AdamW optimizer, and a KV cache for faster step-by-step generation. I trained the required 29.37M- parameter GQA model for 3,000 steps on TinyStories. On the validation data, it reached a perplexity of 5.403 and 0.497 bits per byte (BPB). I checked the implementation with public tests, staff reference files, exact tokenizer and data hashes, cached and uncached comparisons, and a small overfitting test.

1 Implementation and experimental setup

The tokenizer starts with all 256 possible byte values and one special `<|endoftext|>` token. It then learns 7,935 fixed pair merges, giving a final vocabulary of 8,192 tokens. Its SHA-256 hash exactly matched the official checksum, which shows that the learned tokenizer was correct. The language model has 8 decoder layers, with normalization before each main block. It uses $d_{\text{model}} = 512$, 8 query heads, head size 64, a SwiGLU hidden size of 1,536, and a context length of 512. The required GQA model uses 4 KV heads. For a fair comparison, the MHA model changes only the number of KV heads from 4 to 8. RoPE is added to queries and keys, and cached decoding uses the correct position offset [3]. The RMSNorm, SwiGLU, and GQA designs follow [4, 5, 6].

I used random seed 2026 and BF16 number format. Each small batch contained 8 sequences, and I combined gradients across 32 small batches. Since every sequence had 512 tokens, each optimizer step used $8 \times 32 \times 512 = 2^{17}$ tokens. AdamW used $\beta = (0.9, 0.95)$, weight decay 0.1, and a maximum gradient norm of 1.0 before clipping. The learning rate rose in a straight line for 150 steps until it reached 3×10^{-4} , then followed a cosine curve down to 3×10^{-5} . The GQA run saw 393,216,000 tokens during 3,000 steps on one NVIDIA H200 with PyTorch 2.13.0+cu126. Training took 2,011.7 seconds (0.559 h), averaged 195,467 tokens/s, and used at most 2.65 GiB of allocated GPU memory. The MHA model trained for 1,000 steps on an NVIDIA H100. Because the two training runs used different GPU models, I do not use their training speeds as a fair GQA-versus-MHA comparison. I ran the cache tests for both checkpoints one after the other on the same H200.

2 Tokenizer analysis

Table 1 tests shorter and longer prefixes of the same learned merge list. I did not train a new tokenizer for every vocabulary size. A larger vocabulary compressed the text better: bytes per token almost doubled from 2.480 to 4.895, while tokens per word dropped from 2.040 to 1.034. The tradeoff was speed. Encoding slowed from 0.753 to 0.546 MB/s because the tokenizer had more merge rules to check. Learning the full vocabulary from the fixed 46,906-byte data shard took 0.234 s on ICE, which is below the published limit of 0.853 s.

Table 1: Tokenizer prefix analysis on the fixed 44,843-byte validation span.

Vocab	Tokens	Bytes/token	Tokens/word	Encoding MB/s
512	18,080	2.480	2.040	0.753
1,024	13,234	3.388	1.494	0.630
2,048	10,710	4.187	1.209	0.577
4,096	9,477	4.732	1.070	0.552
8,192	9,161	4.895	1.034	0.546

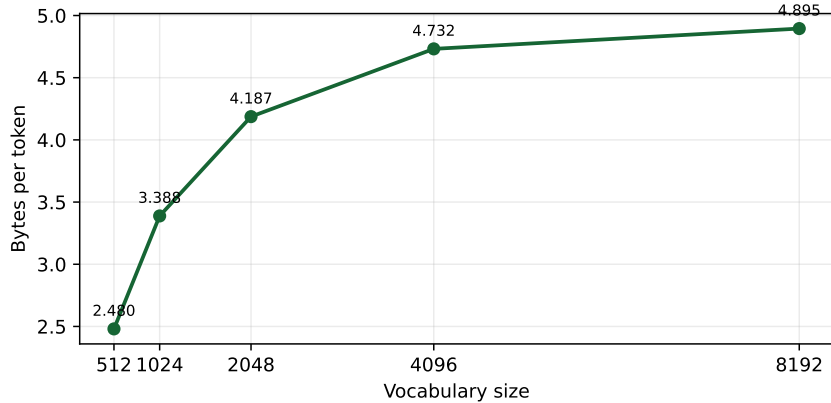


Figure 1: Vocabulary size compared with bytes per token. The extra compression becomes smaller after 4,096 tokens.

3 GQA training and final evaluation

Figure 2 shows that training stayed stable. Training loss and validation NLL stay close together, so there is no clear sign of overfitting near the end. After the noisy beginning, the gradient norm before clipping settles near 0.53. Speed stays around 190–202k tokens/s most of the time. The short drop near step 900 did not happen with any unusual change in loss or gradient size, so it was most likely caused by normal changes in the computer system rather than the model.

I evaluated the final checkpoint on a fixed piece of validation text containing 9,161 tokens, or 44,843 bytes. Each of the 9,160 next-token answers was scored exactly once. The results are shown in Table 2. Token perplexity is only fair to compare when the tokenizer stays the same, because different token boundaries change both the number and difficulty of the predictions. Bits per byte (BPB) is more useful across tokenizers because it divides the total negative log-likelihood by the number of original bytes:

$$\text{BPB} = \frac{\sum_i -\log p(x_i | x_{<i})}{N_{\text{bytes}} \log 2},$$

This makes BPB a more meaningful way to compare different tokenizations of the same text.

Table 2: Final GQA evaluation at step 3,000. Lower is better for all three loss metrics.

Mean NLL	Token PPL	Bits/byte	Target tokens
1.68699	5.40318	0.49715	9,160

For text generation, I used the four provided prompts and generated 128 new tokens for each one. The settings were temperature 0.8, top- $k = 50$, top- $p = 0.95$, and seeds 2026–2029. The complete, unchanged outputs are in Appendix A. Each sample starts with readable TinyStories-style writing, but two problems appear more than once. **(1) The model does not always stop or stay on topic.** Three samples produce `<|endoftext|>` and then begin a new, unrelated story because generation continues until it reaches the fixed token limit. Some samples also stop in the middle of a sentence. **(2) The model loses track of characters and causes.** For example, the robot is called “the robot,” “it,” and “he.” In the trumpet story, “girl” later changes to “his.” The box story forgets about the stuck box and suddenly moves to opening a house door and finding toys. These mistakes suggest that the model can write short, grammatical parts but has trouble remembering the full story over a longer distance.

4 MHA versus GQA and KV-cache behavior

In the controlled comparison after 1,000 steps, MHA has 2,097,152 more parameters, or 7.14% more than GQA. It also has slightly lower validation NLL and perplexity, as shown in Table 3. In other words, the extra KV projection weights gave MHA a small quality advantage when both models saw the same number of training tokens. GQA still reached almost the same quality while using fewer parameters.

GQA TinyStories Training

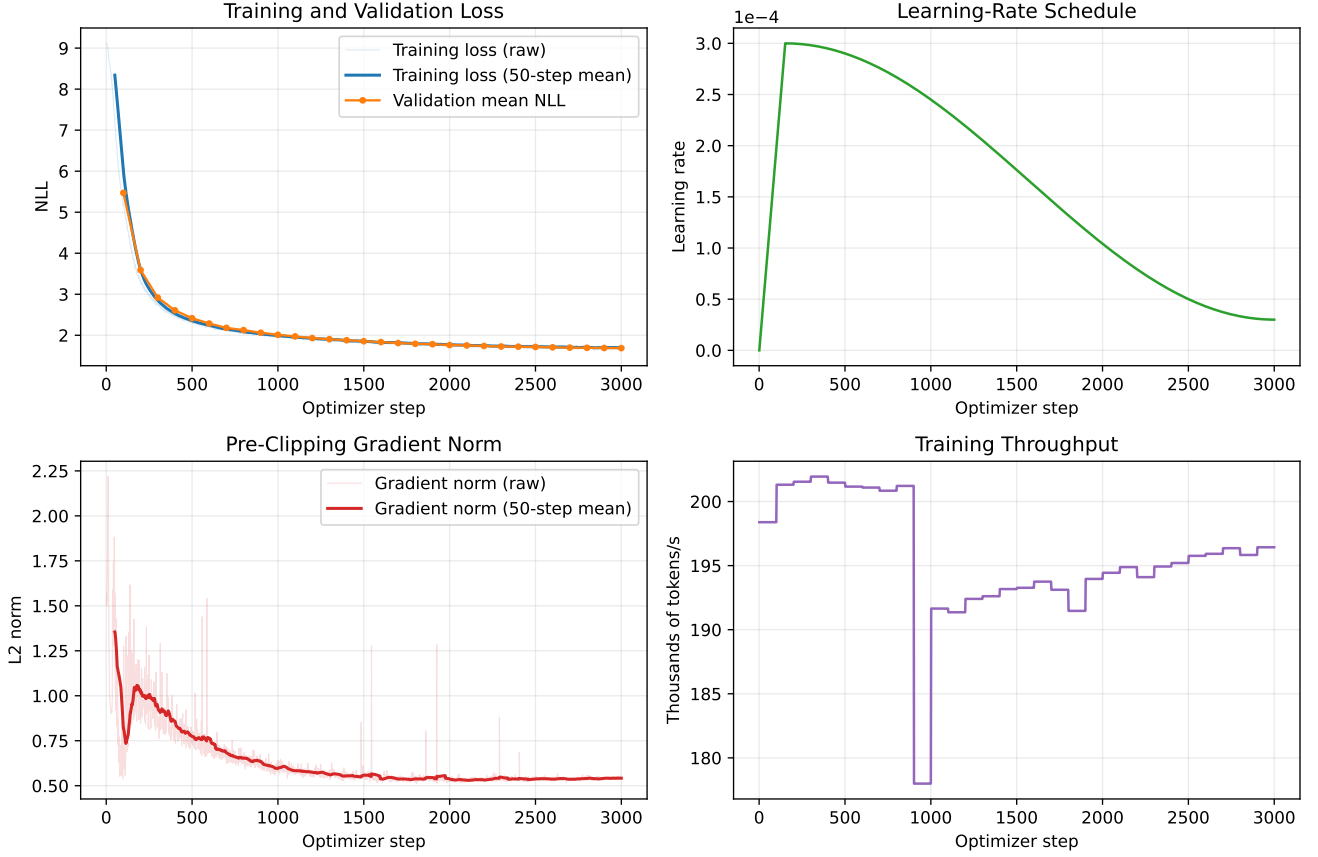


Figure 2: GQA training results. The light lines show raw loss and gradient norm, while the dark lines show 50-step moving averages. Validation NLL was measured every 100 steps.

Table 3: Controlled validation comparison at step 1,000 (131,072,000 tokens seen).

Model	Parameters	Validation NLL	Validation PPL
GQA (8 Q, 4 KV)	29,368,832	2.01174	7.47628
MHA (8 Q, 8 KV)	31,465,984	1.99206	7.33059

The KV cache saves old keys and values so the model does not need to compute them again for every new token. If there are L layers, batch size B , T cached tokens, H_{kv} KV heads, head size d_h , and s bytes per number, storing both keys and values needs

$$M_{KV} = 2LBTH_{kv}d_h s.$$

MHA gives every query head its own key and value heads, so $H_{kv} = H_q$ and $M_{KV}^{\text{MHA}} = 2LBTh_{\text{model}}s$. GQA shares key and value heads across groups. If there are G groups, its cache uses only G/H_q as much memory as MHA. MQA goes further by using one KV head, so it uses $1/H_q$ as much. In this experiment, $L = 8$, $T = 256$, $d_h = 64$, and each BF16 number uses $s = 2$ bytes. Therefore, 4-head GQA uses exactly half as much cache memory as 8-head MHA.

The measured speed difference is small. With a model this size, cached generation starts many tiny GPU jobs, one token at a time. Uncached generation repeats more math, but it does that work in larger matrix operations that use the GPU well. On an H200, the cost of starting many small jobs can cancel out some of the cache’s saved math. The clearest result is memory use: GQA cuts KV-cache storage in half. I checked correctness with staff reference logits, identical cached and uncached tokens, correct RoPE positions, and tests showing that after the prompt, only new tokens are sent through the key and value projections.

Table 4: H200 greedy-decoding benchmark (128-token prompt plus 128 generated tokens).

Model	Batch	Cache	Cached tokens/s	Speedup
GQA	1	2 MiB	198.42	0.996×
GQA	32	64 MiB	6,230.97	1.020×
MHA	1	4 MiB	210.04	0.995×
MHA	32	128 MiB	6,631.57	1.032×

5 Required conceptual analysis and debugging evidence

Causal masking and parallel training. During teacher-forced training, the whole correct input sequence is already available. This means the model can calculate Q, K, and V for every position at the same time. However, a token must not look ahead at future answers. A triangular mask solves this problem by setting every future attention score ($j > i$) to $-\infty$ before softmax. Those future positions then get probability zero. All rows are still calculated together, but position i can only use positions up to i , so future information cannot leak into training.

AdamW versus L2 under Adam. Standard L2 regularization adds $\lambda\theta$ to the gradient before Adam calculates its moving averages and per-parameter scaling. As a result, the amount of shrinking can change based on Adam’s internal statistics. AdamW keeps weight decay separate and directly applies $\theta \leftarrow (1 - \eta\lambda)\theta$ [7]. I used parameter groups so matrix weights receive weight decay, while the one-dimensional RMSNorm scale parameters do not.

A real bug I fixed. At first, the test for pausing and restarting the optimizer failed, even though the saved state loaded without an error. By checking where the tensors were stored in memory, I found that PyTorch’s `load_state_dict` left the loaded `exp_avg` and `exp_avg_sq` tensors sharing storage with the original optimizer state. When the original optimizer took another step, it secretly changed the state that the restarted optimizer was about to use. This made their next parameters different. Loading a `deepcopy(state_dict)` gave the restarted optimizer its own separate tensors and fixed the problem. After that, both the parameter-group restart test and the full uninterrupted-versus- restarted training test passed. The second test also confirmed that the random-number and data- generator states were restored correctly.

6 Conclusion

This project completes the required tokenizer and language-model training process from beginning to end. The final GQA model is close to the published reference quality while using only half as much KV-cache memory as MHA. The experiments also show that doing less math does not always make a program much faster: for a small model generating one token at a time on a powerful GPU, the extra overhead of many small GPU jobs can hide the speed benefit.

A Full generated samples

A.1 Prompt: Once upon a time (seed 2026)

Once upon a time, a little girl named Lucy went to the garden. She saw her friend Tom. Tom was eating some broccoli. Lucy was jealous. She wanted a big broccoli and wanted a small broccoli. Lucy said, "Please, can we both have a small piece of broccoli?" Tom nodded and said yes. They shared the broccoli and they were happy. From that day on, Lucy and Tom shared the broccoli with each other. <|endoftext|> One day, a little girl named Lily went to the park with her mom. They saw a big tree with a long string hanging from it. Lily wanted to get the string, so she ran to the tree and got her long necklace

A.2 Prompt: Lily found a strange box (seed 2027)

Lily found a strange box and she wondered what was inside. She tried to open it, but it was stuck. She tried and tried to open it, but it wouldn't budge. She felt sad and frustrated. Suddenly, she had an idea. She ran to the house and opened the door. She opened it and saw many toys inside. They were not normal to play with anymore. They were not so bad after all. She was so happy, she hugged the toy and played with it all day. <|endoftext|> Sam and Ben were playing with their toys. Sam had a big truck and Ben had a small car. They liked to make the car go fast on the floor and pretend it was their car. Sam had an idea. He thought it was a new game. He wanted to make it go

A.3 Prompt: The little robot looked at the moon (seed 2028)

The little robot looked at the moon and said, "This place is too dark. You should go back home." The robot was very sad. It wished he had stayed home with its mommy and daddy. Sadly, it was too late. The robot was gone forever. <|endoftext|> Once upon a time, there was a girl who wanted to learn how to play the trumpet. She went to her room and grabbed her trumpet. She started to play a song to it. She thought it was funny and made her feel strange. Suddenly, a gust of wind blew and the trumpet went flying out of his room. It was blocking the whole room! The girl was very sad and she

A.4 Prompt: Ben and Mia went into the forest (seed 2029)

Ben and Mia went into the forest. They saw many trees and flowers. They saw a fountain and a squirrel. They saw a pond with ducks and a squirrel on them. They saw a rabbit who was sleeping on a tree. They saw a hole in the ground and ran towards it. They wanted to hide in it, but they heard a voice. "Hey, you two, get out of there! It's my home! It's my home!" It was a small bird who had seen them from the hole in the fence. The bird looked sad and hungry. She saw Ben and Mia and felt sorry for them. She decided to be gentle and

References

- [1] R. Sennrich, B. Haddow, and A. Birch. Neural Machine Translation of Rare Words with Subword Units. ACL, 2016.
- [2] A. Vaswani et al. Attention Is All You Need. NeurIPS, 2017.
- [3] J. Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021.
- [4] B. Zhang and R. Sennrich. Root Mean Square Layer Normalization. NeurIPS, 2019.
- [5] N. Shazeer. GLU Variants Improve Transformer. 2020.
- [6] J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP, 2023.
- [7] I. Loshchilov and F. Hutter. Decoupled Weight Decay Regularization. ICLR, 2019.